NAME :- RISHABH TRIPATHY

REG NO :- 25BAI11067

BRANCH :- B.TECH CSE (AI & ML)

YEAR :- FIRST

COURSE CODE:- CSA2001

DATE :- 31st OF MARCH , 2026

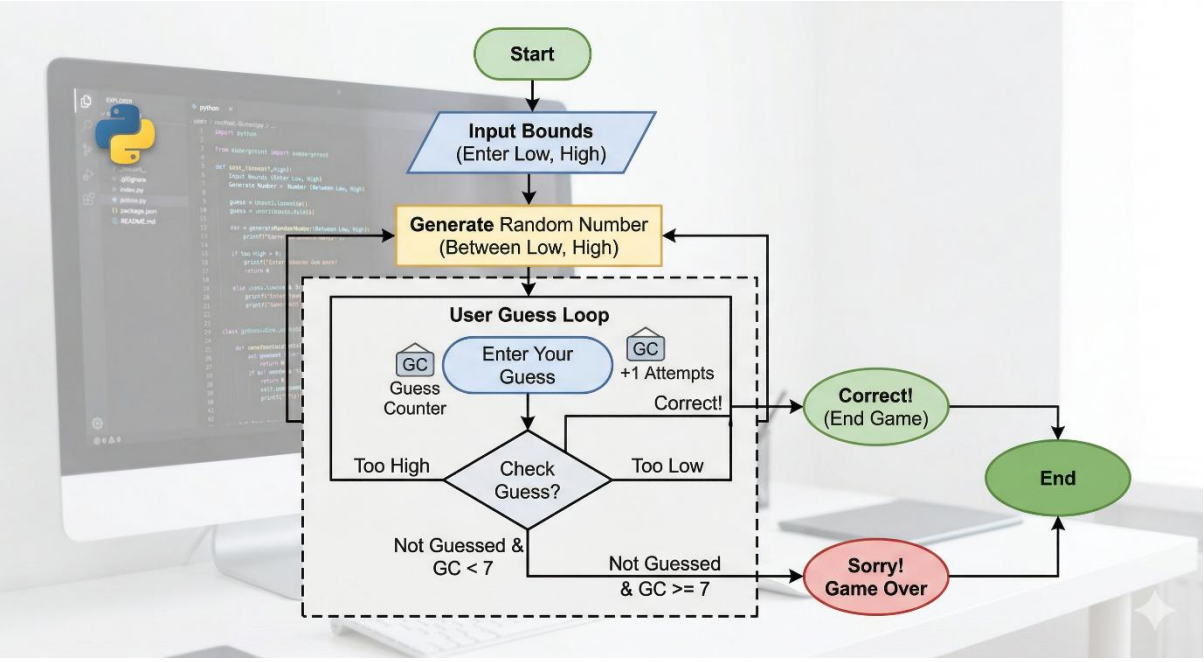
INTRODUCTION

The **Number Guessing Game** is a classic interactive computational challenge that serves as a fundamental exercise in logic, user input handling, and flow control within the Python programming environment. The primary objective of the application is to challenge the user's deductive reasoning by tasking them with identifying a randomly generated integer within a user-defined range.

By leveraging Python's built-in random module, the program ensures a unique experience in every session, maintaining high replayability. From a technical standpoint, the project demonstrates the practical application of conditional statements (**if-elif-else**), loop structures (**while loops**), and dynamic variable tracking. It also incorporates a "limited attempts" mechanic, adding a layer of gamification and urgency. This project not only showcases the ability to manage user data but also highlights the importance of providing real-time feedback—guiding the player with "higher" or "lower" hints—which is a cornerstone of intuitive software design and user experience.

Flowchart

A visual representation of how the code works is highly valued in reports.



1. Objectives

List what the program aims to achieve from a technical and user perspective.

- To generate a random integer within a user-specified range.
- To implement a limited-attempt system (7 chances).
- To provide real-time feedback (Too High/Too Low) to guide the user.

2. System Requirements

Define the environment needed to run your code.

- **Language:** Python 3.x
- **Libraries:** random (Built-in)
- **Operating System:** Windows/macOS/Linux

Features & Functionality

The Number Guessing Game is designed to be a lightweight yet engaging console application. Below are the core functional attributes that define the project:

1. User-Defined Dynamic Range

Unlike static guessing games, this program allows the player to input their own **Lower Bound** and **Upper Bound**.

- **Functionality:** The program uses int(input()) to capture these boundaries.
- **Benefit:** This gives the user control over the difficulty level. A range of 1–10 is easy, while 1–1,000 creates a high-stakes challenge.

2. Randomized Secret Number Generation

At the heart of the game is the random.randint() function.

- **Functionality:** Once the bounds are set, the CPU picks a secret integer n such that $low \leq n \leq high$.
- **Benefit:** Ensures that no two games are ever the same, providing infinite replayability.

3. Iterative Guessing Loop with Attempt Limit

The game implements a strict "7-chances" rule to prevent endless guessing and add a layer of strategy.

- **Functionality:** A while loop runs as long as the guess counter (gc) is less than the allowed chances (ch).
- **Benefit:** This introduces a "Lose Condition," which is essential for making it a true game rather than just a utility.

4. Intelligent Feedback System (Hints)

To assist the player, the program provides real-time logical feedback after every incorrect guess.

- **Too High:** If the guess exceeds the target, the user is prompted to try a lower number.
- **Too Low:** If the guess is below the target, the user is prompted to try a higher number.

- **Benefit:** This allows the user to use a **Binary Search** strategy, narrowing down the possibilities logically rather than guessing blindly.

5. Success and Failure States

The program clearly defines the end of the session:

- **Win State:** Triggered by the break statement when guess == num. It displays the total number of attempts taken.
- **Loss State:** Triggered when the loop finishes without a correct match, revealing the secret number to the user.

Code Implementation

The program is implemented using **Python 3**, leveraging core programming concepts such as modular imports, conditional branching, and iterative loops. Below is a breakdown of the critical components of the source code.

1. Importing Libraries and Initializing Variables

The project begins by importing the random module, which is essential for generating the secret number.

Python

```
import random


low = int(input("Enter the Lower Bound: "))
high = int(input("Enter the Upper Bound: "))
num = random.randint(low, high)
ch = 7  # Total allowed chances
gc = 0  # Guess counter
```

- **Logic:** The randint(low, high) function is used to ensure the secret number is inclusive of the user's defined boundaries.
- **Variable Design:** gc acts as a tracker, while ch acts as a constant limit to define the game's difficulty.

2. The Control Flow (The While Loop)

The heart of the game is a while loop that manages the game state.

Python

```
while gc < ch:
    gc += 1
    guess = int(input('Enter your guess: '))
```

- **Iteration:** The loop continues as long as the user has attempts remaining.
- **Increment:** gc += 1 ensures that every guess is recorded, moving the user closer to the limit.

3. Conditional Logic (Decision Making)

The program evaluates the user's input using an if-elif-else structure to provide immediate feedback.

Python

```
if guess == num:
    print(f'Correct! The number is {num}. You guessed it in {gc} attempts.')
    break
elif gc >= ch and guess != num:
    print(f'Sorry! The number was {num}. Better luck next time.')
elif guess > num:
    print('Too high! Try a lower number.')
elif guess < num:
    print('Too low! Try a higher number.')
```

- **The Win Condition:** If guess == num, the break statement terminates the loop instantly, rewarding the player.
- **The Loss Condition:** The check gc >= ch ensures that if the final attempt is incorrect, the game provides the correct answer before closing.
- **Hint System:** The > and < operators compare the integers to guide the player's next move.

Conclusion

The **Number Guessing Game** project successfully demonstrates the power of Python in creating interactive, logic-driven applications with minimal code. Through the implementation of this game, the core objectives of managing user input, utilizing the random library, and controlling program flow through loops and conditionals were fully realized.

The project highlights how simple mathematical comparisons (<, > , and ==) can be transformed into an engaging user experience. By limiting the player to seven attempts and providing real-time hints, the program balances challenge and guidance, ensuring the game is neither too easy nor impossibly difficult.

Building this application provided a solid foundation in:

- **Logical Structuring:** Designing a loop that handles multiple outcomes (win, loss, or hint).
- **User Experience (UX):** Crafting clear prompts and feedback messages.
- **Version Control:** Successfully managing and documenting code through a GitHub repository